Faculté des Sciences et Techniques de Tanger

Université Abdelmalek Essaadi

Master's End-of-Study Project (PFE)
Master in **[TODO: your master programme name]**

Learning Offensive Navigation Policies on Heterogeneous Attack Graphs

A GNN Approach to Sequential Privilege Escalation in Active Directory

Author:
Nizar Senbati

Supervisor:
[TODO: Supervisor name & title]

Academic Year 2025 – 2026

Acknowledgements

[TODO: Write acknowledgements]

Abstract

AD pentest

Keywords: Graph Neural Network, Heterogeneous Graph Transformer, Active Directory, Privilege Escalation, Attack Graph, Beam Search, Offensive Security, Cross-Environment Generalisation.

Résumé

[TODO: Write French abstract (150 words)]

Mots-clés : Réseau de neurones sur graphe, Transformateur de graphe hétérogène, Active Directory, Escalade de privilèges, Graphe d'attaque, Recherche en faisceau, Sécurité offensive, Généralisation inter-environnements.

Contents

List of Figures

List of Tables

General Introduction

Context

Active Directory is the identity backbone of major enterprises, tho hidden withing the inside network of the company, once the outside defenses are breached (web services, phishing, usb stick ...) the advarsary finds himself able to directly interacte and query the Active Directory Domain Controller itself through ldpa, smb, and other protocols presenting the entierty of the company IT infrastructure data and services to the attacker. A vulnerable AD poses extreme levels of risk making the hardening the AD network environment against an extreme priority.

Credential-based and ACL-based attacks(pass-the-hash, DCSync, Kerberoasting) are among the most impactful vectors in modern breaches. Read teams rely on tools like BloodHound to map the attack surface and identify the vulnerable nodes during an engagement.

Problem Statement

[TODO: Write problem statement paragraph]

Objectives

[TODO: Write objectives paragraph]

Main Contributions

- A complete, modular heterogeneous graph pipeline from raw BloodHound/Certipy JSON to navigable PyTorch Geometric (PyG) `HeteroData` tensors, covering Active Directory Certificate Services (ADCS), Resource-Based Constrained Delegation (RBCD), GPLink, and Security Identifier (SID) history edges.
- A Heterogeneous Graph Transformer (HGT)-based navigation policy trained on Game of Active Directory (GOAD) traces and evaluated on the entirely unseen INLANEFREIGHT enterprise environment, demonstrating generalisation from a 351-node lab to a 4 000-node network.

- Empirical evidence that the learned policy recovers paths with lower step-cost than BloodHound's shortest-path baseline on the same graph.

Report Outline

[TODO: Write report outline (one sentence per chapter)]

Chapter 1

Background and Related Work

1.1 Active Directory: Architecture and Security Model

1.1.1 Core Objects and Structure

[TODO: Write AD structure overview]

1.1.2 Authentication Protocols

[TODO: Write authentication section]

1.1.3 Trust Relationships

[TODO: Write trust relationships section]

1.2 Privilege Escalation in AD Environments

1.2.1 Offensive ACL Abuse

[TODO: Write ACL abuse section]

1.2.2 DCSync Attack

[TODO: Write DCSync section]

1.2.3 ADCS Attack Paths

[TODO: Write ADCS section]

1.2.4 SID History Abuse and RBCD

[TODO: Write SIDHistory and RBCD section]

1.3 Existing Tools and Their Limitations

1.3.1 BloodHound and SharpHound

[TODO: Write BloodHound/SharpHound section]

1.3.2 bloodhound-python

[TODO: Write bloodhound-python limitations]

1.3.3 Other Automated Attack Path Tools

[TODO: Write related tools survey]

1.4 Graph Neural Networks

1.4.1 Message Passing and Neighbourhood Aggregation

[TODO: Write GNN fundamentals]

1.4.2 Graph Convolutional Network (GCN)

[TODO: Write GCN section]

1.4.3 Heterogeneous Graph Transformer (HGT)

[TODO: Write HGT section]

1.4.4 PyTorch Geometric and HeteroData

[TODO: Write PyG section]

1.5 Summary and Positioning

[TODO: Write summary and positioning]

Chapter 2

System Architecture and Design

2.1 System Overview

[TODO: Write system overview and include pipeline diagram]

2.2 Data Collection and Preprocessing

2.2.1 Input Sources

[TODO: Write input sources section]

2.2.2 `clean_bloodhound.py`: Filtering and Multi-Domain Accumulation

[TODO: Write clean_bloodhound.py section]

2.2.3 `merger.py`: Normalisation and Merging

[TODO: Write merger.py section]

2.2.4 `stitcher.py`: ADCS Injection and Domain Detection

[TODO: Write stitcher.py section]

2.3 Heterogeneous Graph Construction

2.3.1 Node and Edge Type Schema

[TODO: Write node/edge schema — include a table]

2.3.2 Direction Convention

[TODO: Write direction convention section]

2.3.3 Edge Extraction Details

[TODO: Write edge extraction details]

2.3.4 PyG HeteroData Encoding

[TODO: Write PyG encoding section]

2.4 Navigation Model

2.4.1 HGT Encoder

[TODO: Write HGT encoder section]

2.4.2 Policy Head

[TODO: Write policy head section]

2.4.3 Loss Function and Step Cost

[TODO: Write loss function section]

2.5 Beam Search Navigation

2.5.1 Algorithm

[TODO: Write beam search algorithm description — consider an Algorithm2e pseudocode block]

2.5.2 Terminal State: DCSync

[TODO: Write terminal state section]

2.5.3 Cross-Environment Inference

[TODO: Write cross-environment inference section]

2.6 Audit Module

[TODO: Write audit module section]

Chapter 3

Implementation

3.1 Technical Stack

[TODO: Write technical stack section — include a versions table]

3.2 Repository Structure

[TODO: Write repository structure section — include the directory tree as a listing]

3.3 Data Pipeline

3.3.1 Script Responsibilities and Data Flow

[TODO: Write pipeline data flow section — include input/output table]

3.3.2 SharpHound Version Normalisation

[TODO: Write SharpHound normalisation section]

3.4 Training

3.4.1 Training Data: GOAD

[TODO: Write GOAD training data section]

3.4.2 Kaggle Training Notebook

[TODO: Write Kaggle notebook section]

3.4.3 Model Checkpoints

[TODO: Write model checkpoints section]

3.5 Inference Pipeline

[TODO: Write inference pipeline section]

3.6 Graphical Interface

3.6.1 Design and Theme

[TODO: Write GUI design section]

3.6.2 Workflow Integration

[TODO: Write GUI workflow section]

3.6.3 Visualisation

[TODO: Write GUI visualisation section]

Chapter 4

Experiments and Results

4.1 Experimental Setup

4.1.1 Training Environment: GOAD

[TODO: Write GOAD setup section]

4.1.2 Test Environment: INLANEFREIGHT

[TODO: Write INLANEFREIGHT setup section]

4.1.3 Test Environment: GOAD castelblack / kingslanding

[TODO: Write castelblack/kingslanding setup]

4.1.4 Coverage Metric

[TODO: Write coverage metric section — include a summary table of all queries]

4.2 Result 1: wley → Domain Admin on INLANEFREIGHT

[TODO: Write Result 1 — include the key comparison figure]

4.3 Result 2: Cross-Domain Trust Traversal

[TODO: Write Result 2]

4.4 Result 3: Constrained Delegation — castelblack → kingslanding

[TODO: Write Result 3]

4.5 Result 4: Documented Limitation — Cross-Forest Foreign MemberOf

[TODO: Write Result 4]

4.6 Discussion

4.6.1 Model Coverage and Training-Set Ceiling

[TODO: Write coverage discussion]

4.6.2 Expressiveness vs. Transferability: HGT vs. GCN

[TODO: Write HGT vs. GCN discussion]

4.6.3 Pipeline Completeness as a First-Class Contribution

[TODO: Write pipeline completeness discussion]

Chapter 5

Limitations and Future Work

5.1 Data Collection Incompleteness

[TODO: Write data collection limitations section]

5.2 Expanding the Training Knowledge Base

[TODO: Write training knowledge base expansion section]

5.3 Terminal State Learning via Offline Reinforcement Learning

[TODO: Write offline RL future direction section]

5.4 Additional Future Directions

5.4.1 Calibrated Confidence Scoring

[TODO: Write calibrated confidence section]

5.4.2 SharpHound Edge Vocabulary Versioning

[TODO: Write edge vocabulary versioning section]

5.4.3 Derived Attack Primitives

[TODO: Write derived primitives section]

General Conclusion

[TODO: Write general conclusion (1 page)]

Appendix A

Sanitised BloodHound JSON Example

[TODO: Insert sanitised JSON excerpt as a lstlisting]

Appendix B

Heterogeneous Graph Schema

[TODO: Insert full graph schema table]

Appendix C

Key Code Excerpts

Effective Weight Loss Function

```
# eff_weight combines path importance with step discretion cost
eff_weight = path_weight * (1 - step_cost)
```

Listing C.1: Effective weight computation in `prepare_training_examples.py`

[TODO: Add beam search loop and has_dcsync_on excerpts here]

Appendix D

GUI Screenshots

[TODO: Insert GUI screenshots]